Applets and Threading

Introduction

- An **applet** is a Java program that runs in a web browser and is embedded in an HTML page.
- It is designed for dynamic and interactive content in webpages.
- Unlike standalone Java applications, applets do not have a main() method and are controlled by the browser or applet viewer.
- An applet is a Java class that extends the java.applet.Applet class.
- When a user views an HTML page that contains an applet, the code for the applet is downloaded to the user's machine.
- A JVM is required to view an applet. The JVM can be either a plug-in of the Web browser or a separate runtime environment.

Java Application vs Applet

Feature	Java Application	Java Applet
Execution	Runs with main() method	Runs inside a web browser or applet viewer
Environment	Standalone program	Embedded in HTML or launched via viewer
Security	Has full access to system resources	Runs in a sandbox (limited permissions)
User Interface	Typically uses command-line or GUI	Must use AWT/Swing for UI
File Access	Can read/write files freely	Limited file access for security
Network Access	Full access	Can only connect to the host it was loaded from

Applet Life Cycle

- Java Applets go through a **well-defined life cycle**, which is managed by the browser or applet viewer.
- Life Cycle Methods:

- **init()**
- Called once when the applet is first loaded.
- Used for initialization tasks and loading the resources.
- **start()**
- Called every time the applet becomes active or visible.
- Used to start or resume operations like animations.
- **paint(Graphics g)**
- Called to draw the applet's output on the screen.
- Automatically called after init() and start().
- **stop()**
- Called when the applet is no longer visible.
- Used to suspend activities (like threads).
- **destroy()**
- Called when the applet is removed completely from memory.
- Used for cleanup tasks.

Working with Applets

To create and use an applet in Java:

1. Extend the Applet class.
2. Override lifecycle methods: init(), start(), paint(), etc.
3. Compile the Java file.
4. Embed the applet in an HTML file using the <applet> tag or use an applet viewer.

```
import java.applet.Applet;
import java.awt.Graphics;

public class HelloApplet extends Applet {
    public void paint(Graphics g) {
        g.drawString("Hello, Applet!", 50, 50);
    }
}
```

The HTML <APPLET> Tag

- The <APPLET> tag is used in HTML to embed a Java applet:

```
<applet code="HelloApplet.class" width="300" height="150">
Your browser does not support Java applets.
</applet>
```

- Attributes:
- code: Name of the .class file
- width and height: Dimensions of the display area

java.applet Package

- The java.applet package contains classes necessary for creating applets.
- Key Classes:
- Applet: The main class used to create applets; contains the lifecycle methods.

Method	**Description**
init()	Initialization logic
start()	Resume execution
stop()	Pause execution
destroy()	Cleanup before applet unloads
getParameter()	Reads parameters from the HTML <param> tag
showStatus()	Displays messages in the browser’s status bar

Drawing and Graphics in Applets

- The paint(Graphics g) method allows drawing on the applet window. You can use it to render text, shapes, and images.
- Common Graphics Methods:

Method	Description
drawString(String, x, y)	Draws text
drawLine(x1, y1, x2, y2)	Draws a line
drawRect(x, y, w, h)	Draws a rectangle
drawOval(x, y, w, h)	Draws an oval
setColor(Color c)	Sets the drawing color
setFont(Font f)	Sets the font for text

Example

```
import java.applet.Applet;
import java.awt.*;
/*
<applet code="SimpleGraphicsApplet.class" width="300" height="200">
</applet>
*/
public class SimpleGraphicsApplet extends Applet {
    public void paint(Graphics g) {
        g.setColor(Color.BLUE);
        g.drawString("Welcome to Java Applets!", 50, 30);
        g.drawRect(40, 50, 100, 50);
        g.setColor(Color.RED);
        g.fillOval(160, 50, 50, 50);
    }
}
```

How to Run:

1.Save the code in a file named SimpleGraphicsApplet.java.
2.Compile: javac SimpleGraphicsApplet.java
3.Create an HTML file with the <applet> tag.
4.Open it with appletviewer:

Note: Modern IDEs and browsers may not support applets. Use **older JDKs** or **command-line tools** like appletviewer for testing.

- **Advantages of Applets**
  - **Platform Independent**: Applets are written in Java, so they can run on any platform with a Java-enabled browser.
  - **Web Integration**: Applets can be embedded directly into web pages for interactive content.
  - **Security**: Applets run in a restricted environment called a sandbox which prevents unauthorized access to system resources.
  - **No Installation Required**: Users don't have to install anything extra—just load the webpage containing the applet.
- **Disadvantages of Applets**
  - **Browser Support**: Modern browsers no longer support Java applets due to security concerns.
  - **Performance**: Applets may be slower to load, especially over slow networks.
  - **Security Restrictions**: The sandbox restricts file I/O, system access, etc., which limits what an applet can do.
  - **Obsolete Technology**: Java applets are largely outdated and replaced by more modern web technologies (e.g., JavaScript, HTML5, etc.).

Alternatives to Applets Today

- Since applets are deprecated, developers now use:
  - **Java Web Start (also deprecated)**
  - **JavaFX** for rich desktop applications
  - **JavaScript/HTML5** for web interactivity
  - **Swing or JavaFX** for GUI-based Java applications

Threading

- **Thread**: A lightweight subprocess; the smallest unit of execution.
- **Threading** refers to the ability of a program to perform **multiple tasks simultaneously** by dividing the execution into **multiple threads**. Each thread is an independent path of execution.
- **Multithreading**: A programming feature that allows concurrent execution of two or more threads.
- **Concurrency**: Performing multiple tasks at once.
- **Parallelism**: Executing multiple threads at the same exact time (e.g., on multi-core processors).
- Java provides built-in support for multithreading, enabling concurrent execution of two or more parts of a program for maximum utilization of CPU.

Creating Threads in Java

- There are **two main ways** to create threads in Java:
- Method 1: By Extending the Thread class

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running...");
    }

    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start(); // starts the thread and calls run()
    }
}
```

Creating Threads in Java

- Method 2: By Implementing the Runnable Interface

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Runnable thread running...");
    }

    public static void main(String[] args) {
        MyRunnable obj = new MyRunnable();
        Thread t1 = new Thread(obj);
        t1.start();
    }
}
```

Comparison:

Feature	Extending Thread	Implementing Runnable
Inheritance	Cannot extend another class	Can extend any class
Flexibility	Less flexible	More flexible
Best Practice	For simple use	Recommended approach

Thread Life-Cycle in Java

A thread in Java goes through the following **life-cycle stages**:

1. New

•A thread is created using new Thread() but has not started yet.

2. Runnable

•After calling start(), the thread is ready to run and is placed in the **runnable pool**.

3. Running

•When the thread is picked by the thread scheduler, it enters the **running** state and executes the run() method.

4. Blocked/Waiting/Sleeping

•A thread can temporarily stop executing and enter a **non-runnable** state due to:

- •Sleeping (Thread.sleep(ms))
- •Waiting (wait())
- •Blocked on I/O or resource

5. Terminated (Dead)

•The thread completes its execution or is terminated manually.

Thread Life-Cycle Diagram

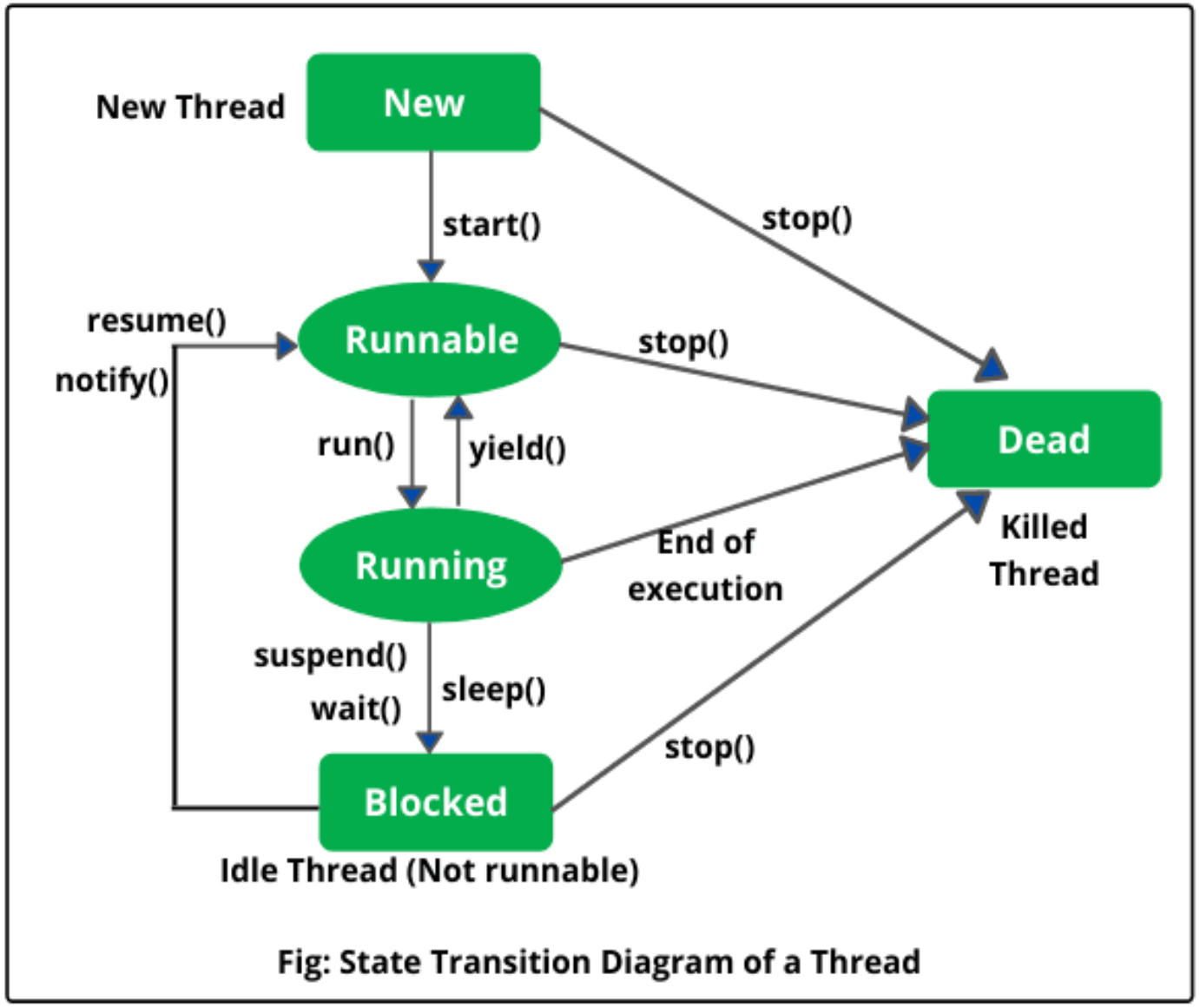


Fig: State Transition Diagram of a Thread

Common Thread Methods

Method	**Description**
start()	Starts the thread and invokes run()
run()	Entry point of the thread
sleep(ms)	Pauses thread for the specified milliseconds
join()	Waits for a thread to die
isAlive()	Checks if thread is alive
setPriority(n)	Sets the thread priority (1 to 10)
yield()	Pauses current thread and allows others to execute

Example: Multi-threaded Program

```
class PrintTask extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(getName() + " prints: " + i);
            try {
                Thread.sleep(3000);
            } catch (InterruptedException e) {
                System.out.println("Interrupted");
            }
        }
    }
    public static void main(String[] args) {
        PrintTask t1 = new PrintTask();
        PrintTask t2 = new PrintTask();
        t1.start();
        t2.start();
    }
}
```

Output:

Thread-0 prints: 1
Thread-1 prints: 1
Thread-0 prints: 2
Thread-1 prints: 2
Thread-0 prints: 3
Thread-1 prints: 3
Thread-0 prints: 4
Thread-1 prints: 4
Thread-0 prints: 5
Thread-1 prints: 5

- **Thread Priorities**
  - Java threads have priorities from **1 (MIN_PRIORITY)** to **10 (MAX_PRIORITY)**.
  - The default priority is **5 (NORM_PRIORITY)**.
  - Higher-priority threads may get more CPU time but **priority is not guaranteed** (depends on OS and scheduler).